CS 650 – Full Stack Application Develop

2-1 Short Paper: Back-End API Framework Comparison

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

July 14, 2026

Back-End API Framework Recommendation

Choosing the right back-end framework is an important decision because it affects how quickly a team can develop an application, how easy the code is to maintain, and how well the application can grow over time (Fowler, 2018). For this project, a small development team must build a CRUD API for a local business journal that serves a metropolitan area of approximately 200,000 people. Editors will use an administrative dashboard to publish articles, manage business profiles, update local events, and maintain digital journal editions, while readers will access this information through a public-facing website. The API must support JSON requests and responses, secure authentication and authorization, and deployment on the AWS Free Tier. Because the team has only four weeks to deliver a minimum viable product (MVP), selecting a framework that matches the team's experience is critical. This paper compares NestJS and Ruby on Rails (API mode), discusses their similarities and differences, and recommends the framework that best meets the project's technical and business requirements.

Project Requirements and Constraints

Several requirements influence the selection of a back-end framework for this project. First, the application must provide a RESTful API that supports create, read, update, and delete (CRUD) operations for articles, business profiles, events, journal editions, and user accounts. The API should receive JSON payloads from the client application and return JSON responses. Security is another essential requirement because only editors and administrators should be allowed to create, update, or delete content. This means the framework should support authentication, authorization, and request validation (Fowler, 2018).

The project also has practical constraints. The development team consists of three developers, including two with strong JavaScript or TypeScript experience and one with Ruby experience. The MVP must be completed within four weeks, leaving little time for learning entirely new technology. In addition, the organization has a limited budget and plans to deploy the application using the AWS Free Tier. Traffic is expected to be modest when the application launches, but the framework should still be capable of supporting future growth without requiring a major redesign.

Framework Overview

NestJS

NestJS is an open-source server-side framework built with TypeScript and designed for developing scalable, maintainable web applications and REST APIs (NestJS, n.d.). It is built on top of Express or Fastify and follows a modular architecture that separates applications into controllers, services, and modules. This structure makes projects easier to organize, improves code reuse, and simplifies maintenance as applications become more complex.

NestJS is commonly used for enterprise applications, REST APIs, GraphQL services, and microservices. Organizations such as Adidas, Roche, and Capgemini have adopted NestJS because of its scalability and structured architecture (NestJS, n.d.).

Ruby on Rails (API Mode)

Ruby on Rails is an open-source framework written in Ruby that emphasizes developer productivity through its "convention over configuration" philosophy (Ruby on Rails, n.d.).

Although Rails is widely known for developing complete web applications, it also offers an API-only mode that allows developers to build RESTful APIs that exchange JSON data. Rails organize applications using the Model-View-Controller (MVC) architecture, which separates business logic, data management, and request handling into different components. This organization promotes maintainability while allowing developers to build applications quickly using built-in tools and conventions (Ruby on Rails, n.d.). Ruby on Rails has powered many successful applications, including GitHub, Shopify, and Airbnb, demonstrating its ability to support both startups and large-scale applications (Ruby on Rails, n.d.).

Common Features

Although NestJS and Ruby on Rails use different programming languages and design philosophies, they provide many of the same capabilities needed for this project.

Both frameworks support:

- RESTful API development using JSON.
- CRUD operations for creating, reading, updating, and deleting resources.
- Database integration through Object-Relational Mapping (ORM) tools.
- Authentication and authorization through well-supported libraries.
- Routing, middleware, validation, and error handling.
- Large developer communities with extensive documentation and third-party packages (NestJS, n.d.; Ruby on Rails, n.d.).

Because of these shared capabilities, either framework could successfully meet the technical requirements of the local business journal.

Framework Comparison

Feature	NestJS	Ruby on Rails
Primary Language	Best Fit for This Scenario	Ruby
Project Organization	Modules, controllers, and services	Model-View-Controller (MVC)
Built-in Features	Core framework with optional libraries	Extensive built-in functionality
Community	Large JavaScript/TypeScript ecosystem	Mature Ruby ecosystem
Deployment	Lightweight Node.js deployment on AWS	Easily deployable but typically requires more resources
Scalability	Excellent for modular applications and microservices	Proven scalability for large web applications
Best Fit for This Scenario	Strong alignment with team skills	Strong framework but less aligned with developer experience

One of the biggest differences between these frameworks is the balance between built-in functionality and flexibility. Ruby on Rails includes many features immediately after installation, such as database migrations, generators, routing, and Active Record. These built-in capabilities allow developers to produce applications quickly with relatively little configuration (Ruby on Rails, n.d.). NestJS takes a different approach by providing a structured foundation while allowing developers to select additional libraries based on project needs. Although this may require slightly more initial setup, it gives teams greater flexibility and avoids including unnecessary components (NestJS, n.d.).

Both frameworks benefit from large and active communities. Rails have been widely used for nearly two decades and offer thousands of mature libraries. NestJS benefits from the enormous JavaScript and TypeScript ecosystem, allowing developers to integrate modern Node.js packages using npm. The popularity of JavaScript also makes it easier to find documentation, tutorials, and community support (NestJS, n.d.; Ruby on Rails, n.d.). Deployment is another important consideration. Both frameworks can be hosted on AWS Free Tier resources. However, Node.js applications created with NestJS generally require fewer server resources and integrate naturally with many cloud deployment platforms. This can reduce hosting costs and simplify deployment for a small organization operating with a limited budget (Newman, 2021).

From a scalability perspective, both frameworks can support applications beyond the initial launch. Ruby on Rails has demonstrated its ability to power high-traffic platforms, while NestJS was specifically designed with modular architecture and microservices in mind. This flexibility makes NestJS well-suited for applications that may expand over time (Newman, 2021). Developer experience is perhaps the most significant factor in this scenario. Two of the three developers already have experience with JavaScript and TypeScript. Selecting NestJS allows the team to build on its existing knowledge instead of learning a new programming language under a tight deadline. This reduces onboarding time, increases productivity, and lowers the overall project risk. While the Ruby developer may need to become familiar with NestJS, the framework's clear documentation and organized structure make the learning process manageable (NestJS, n.d.).

Recommendation

After evaluating both frameworks, NestJS is the strongest choice for this project. Both NestJS and Ruby on Rails satisfy the technical requirements by supporting REST APIs, JSON communication, CRUD operations, authentication, database integration, and deployment on AWS. However, selecting the best framework requires considering the project's timeline, team experience, and long-term goals rather than technical features alone (Fowler, 2018).

The most important advantage of NestJS is its alignment with the development team's existing skills. Because two of the three developers already work with JavaScript or TypeScript, they can begin development immediately without investing valuable time in learning a new programming language. Given the four-week deadline, this advantage significantly reduces development risk and improves the likelihood of delivering a successful MVP on schedule. NestJS also provides a modular architecture that simplifies maintenance and future expansion. As the local business journal grows, new features such as mobile applications, analytics, or additional APIs can be added without requiring major architectural changes. Furthermore, the Node.js ecosystem provides access to thousands of well-maintained packages that simplify authentication, validation, logging, and database connectivity (Newman, 2021).

Based on the available evidence and the specific project requirements, NestJS provides the best balance of developer productivity, maintainability, scalability, and cost-effectiveness. While Ruby on Rails remains an excellent framework, NestJS is the more practical choice because it better matches the team's experience and the project's timeline, increasing the likelihood of delivering a reliable application within budget.

References

Fowler, M. (2018). *Patterns of enterprise application architecture*. Addison Wesley Professional.

NestJS. (n.d.). *NestJS documentation*. https://docs.nestjs.com

Newman, S. (2021). *Building microservices* (2nd ed.). O'Reilly Media.

Ruby on Rails. (n.d.). *Ruby on Rails Guides*. https://guides.rubyonrails.org